情報科学演習 D

課題 4 レポート

【担当教員】　桝井 晃基，Olivier Nourry　教員

【提出者】　向井 大貴　(09B23081)
ソフトウェア科学コース・3 年
u230984j@ecs.osaka-u.ac.jp

【提出日】　2026 年 1 月 30 日

1　システムの仕様

`Compiler.run(String)` に意味解析器を実装する．第一引数に与えられた ts ファイルを読み込み, CASL のコードを生成する．問題がなかった場合は'OK' という文字列を返し, 意味的・構文的に問題があった場合は以前の課題に準じたエラーメッセージを返す．

2　課題達成の方針と設計

前提

コンパイラでは, これまでの課題で実装した AST をスタート地点として, 目的コードへの変換を行う．これまでの課題で, 構文や意味的な正しさは検証済みであり,AST も正しく構築されていることが前提となる．また, 意味解析機において変数名のユニーク化を行っているため, 同じ名前の変数が異なるスコープで宣言されている場合でも, 変数名の衝突を気にする必要はないことも前提とする．

設計の方針と目的

実装するコンパイラにおいては, 課題で示された要求仕様に加え, 以下の設計方針を採用する．

1. CASL 以外のアセンブリ言語への拡張を見据えた設計とする．
2. 大域的最適化と局所的最適化を区別した設計とする．
3. 言語機能の拡張に対応しやすい設計とする．

CASL 以外のアセンブリ言語への拡張を見据えた設計　今回の課題では, 最終的な出力コードとして CASL を生成することが求められている．実際のコンパイラでは, ターゲットとするアーキテクチャが x86 や arm など複数存在し, それぞれに対応したコード生成が必要となる．そのため, より汎用的な設計を行う必要がある．コード生成部分を抽象化し, アーキテクチャ依存の部分と共通部分を明確に分離することで, 将来的に他のアセンブリ言語への対応を容易にすることを目指す．

大域的最適化と局所的最適化の区別　コンパイラにおける最適化は, 大域的最適化と局所的最適化に分けられる．大域的最適化は, プログラム全体を対象として行われる最適化であり, 例えばデッドコードの削除やループの展開などがある．一方, 局所的最適化は, 基本ブロック単位で行われる最適化であり, 例えば定数畳み込みや代入の削除などがある．大域的最適化と局所的最適化は, 対象とする範囲や手法が異なるため, これらを区別した設計とすることで, 将来的な拡張や保守性を向上させる．

言語機能の拡張に対応しやすい設計　今回の課題で扱う言語は,Pascal のサブセットであり, 基本的な構文要素のみを含んでいる．例えば, 副プログラムを入れ子にできないなどの制約がある．この制約により, 出力するコードの生成などは用意になるが, 高度なコンパイラを実装してみたいという観点から, できるだけ言語機能の拡張に対応しやすい設計を目指す．

設計

以上の方針を踏まえ, コンパイラの設計を行った．まず, コンパイラの全体構成を図 1 に示す．

コンパイラは,AST を直接目的コードに変換するのではなく, 中間表現 (IR: Intermediate Representation) を経由して変換を行う．IR は, 高水準の抽象度を持つ中間コードであり, ターゲットとするアーキテクチャに依存しない形でプログラムの意味を表現する．今回は, 二種類の中間表現を用意し, それぞれの役割を区別することで目的の実現を目指す．

まず, 最初の中間表現は IR と呼び,AST から変換される．IR は, 分岐やループなどの処理をジャンプを用いて表現するなど, 元のプログラムよりは低水準で表現する．大域的最適化は, デッドコードや冗長な計算の削除など, プログラム全体を対象とした最適化を行うものであり, 特定のアーキテクチャに依存しない形で実装することが可能である．そのため,IR はターゲット非依存の中間表現として設計し, 大域的最適化をこの段階で行う．そのため IR では, 値がメモリやレジスタに格納されていることを意識せずに, 変数や定数, 演算などを扱うようにする．

2 つ目の中間表現は TIR (Target-dependent Intermediate Representation) と呼び,IR から変換される. TIR は, レジスタやメモリアドレスなど, ターゲットアーキテクチャに依存した要素を含む中間コードである. TIR の命令列は, サブルーチンの呼び出しや引数の受け渡しなども, ターゲットに依存した形で扱う. なお, レジスタ割り当て後に行いたい局所最適化も存在し, これらは Assembly に対して処理を行う. また, ターゲットアーキテクチャの命令セットに近い形で表現されているため, 局所的最適化をこの段階で行う. 局所的最適化では, レジスタ割り当てや命令の組み換えなど, ターゲットに依存した最適化を行う. TIR では, レジスタは数に制限のない仮想レジスタとして扱い, 最終的なコード生成時に物理レジスタへ割り当てる.

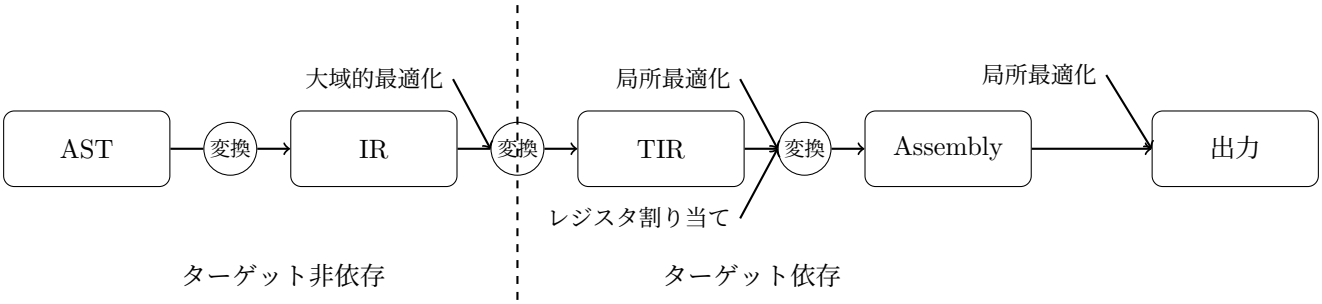


図 1: コンパイラの設計概要

3 詳細設計

3.1 出力する CASL

CASL ライブラリの利用

出力する CASL コードでは, 乗除算や剰余算, 入出力などの基本的な機能を授業資料に示された CASL ライブラリに依存する.

レジスタの予約

今回の課題で対象とする CASL では GR0 から GR8 までの 9 つのレジスタが存在し,GR8 はスタックポインタとして予約されている. そのため, 汎用レジスタとして GR0 から GR7 までの 8 つを使用することができるが, 今回は更にいくつかのレジスタを予約することで, コード生成を容易にしている.

利用する CASL ライブラリでは, 二項演算のサブルーチンでは, 引数として GR1 と GR2 を使用し, 結果を GR1 もしくは GR2 に格納することが規定されている. そのため,GR1 と GR2 はライブラリの呼び出し時には使用される可能性があるため, 開けておく必要がある. そのため,GR1 はそれ専用に予約し,GR2 は一時的に使用する一時レジスタとして予約することにした. 一時レジスタは, メモリアドレスの計算など, 複数の命令に跨らない形で使用することを想定しているため, ライブラリの呼び出しには支障しない.

GR6,GR7 については, ライブラリ内で文字列を扱う際のバッファなどで使用するため, 特定の値を格納しておくことが規定されている. しかし, コード全体を見れば文字列を扱う回数は限られており, そのために常にこれらのレジスタを予約しておくのは非効率である. そのため, これらのレジスタは汎用的に使えるように解放しておき, ライブラリの呼び出し時には内容を退避したうえで必要な値をメモリからロードすることとする.

今回出力する CASL コードでは, フレームを用いて変数を管理する. そのため, フレームポインタとして GR5 を予約する.

フレームの設計

関数呼び出し時における変数の管理にはフレームを用いる. フレームは, 関数ごとに確保されるメモリ領域であり, ローカル変数や引数, 戻りアドレスなどを格納するためにスタック上に確保される. フレームの構造を図 2 に示す.

フレームには, 引数, 静的リンク, リターンアドレス, 動的リンク, ローカル変数の順に格納される. 引数と静的リンク, リターンアドレスは, 呼び出し元で計算されるため, 呼び出し前にフレームにプッシュされる. 動的リンクは, 呼び出し先で計算されるため, 呼び出し先でフレームにプッシュされる. また, フレームポインタ (FP: Frame Pointer) は,

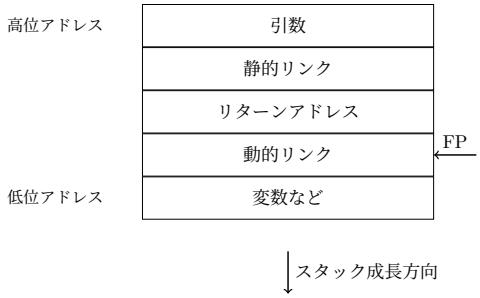


図 2: フレームの構造

動的リンクの位置を指すように設定され, 変数領域の大きさ分だけ SP を減算することで, ローカル変数領域が確保される.

なお, 静的リンクとは, ネストした関数呼び出しにおいて, 外側の関数のフレームへのポインタを指すものである. また, 動的リンクとは, 呼び出し元の関数のフレームへのポインタを指すものである.

変数へのアクセス

変数へのアクセスは, フレームポインタを基準としてオフセットを計算することで行う. 例えば, ある関数内で宣言されたローカル変数へのアクセスは,FP から変数領域のオフセットを計算し, そのアドレスに対して読み書きを行うことで実現する. 副プログラムの引数へのアクセスも同様に,FP を基準としてオフセットを計算することで行う. もし, ネストした関数内で外側の関数の変数にアクセスする必要がある場合, 静的リンクを辿って目的のフレームポインタを取得し, そこからオフセットを計算することで実現する.

3.2 IR

オペランド

IR で定義するオペランドを表 1 に示す.

命令

IR のすべての命令は,SSA(Static Single Assignment) 形式を満たすように設計する. SSA 形式とは, 各変数がプログラム中で一度だけ代入されることを保証する形式であり, これによりデータフロー解析や最適化が容易になる. IR は, 機械語非依存であるため設計上の自由度があり,SSA 形式を満たすように設計することが可能である.

命令名	オペランド	説明
BinaryOp	temp(dst), src1, src2	src1 と src2 の二項演算を行い、その結果を dst に格納する
Call	label(target), args[]	target を args を引数として呼び出す
Jump	label(target), cond	target にジャンプ。無条件,cond=True,False の条件付きが選べる
Load	temp(dst), var(src)	src から値を読み込み、dst に格納する
LoadIndexed	temp(dst), var(base), offset	base の offset の位置から値を読み込み、dst に格納する
Move	temp(dst), src	src の値を dst にコピーする
Read	var(dst)	入力から値を読み込み、dst に格納する
Store	var(dst), src	src の値を dst に格納する
StoreIndexed	var(base), offset, src	src の値を base の offset の位置に格納する
UnaryOp	temp(dst), src	src の単項演算を行い、その結果を dst に格納する
Write	src	src の値を出力する

3.3 TIR

命令

TIR の命令は, ターゲットアーキテクチャに依存した形で設計する. 基本的には,CASL の命令セットに対応する形で設計するが, 一部の高水準な操作については, 複数の CASL 命令に展開されるような命令も設計する. また, レジスタ

オペランド名	説明
Const	定数値
Variable	変数
Temp	一時変数
Label	ラベル

表 1: IR オペランド一覧

オペランド名	説明
AddressOperand	アドレスやラベル
FrameSlot	フレームスロット
NumericImmediate	数値リテラル
StringImmediate	文字列リテラル
VirtualRegister	仮想レジスタ

表 2: TIR オペランド一覧

割り当てが完了するまで決まらないフレームの生成処理などは,`Prologue`,`Epilogue` 命令としてまとめて設計する.

命令名	オペランド	説明
Add	vreg(op1), data(op2)	op1 = op1 + op2
And	vreg(op1), data(op2)	op1 = op1 & op2
Call	address(target)	target を呼び出す
Cmp	vreg(op1), data(op2)	op1 と op2 を比較する
Epilogue		フレームの後始末を行う
Jmp	address(target)	target にジャンプする. 正負, ゼロ, 非ゼロ, オーバーフローの条件も指定可能
Lad	vreg(dst), data(src)	src の値を dst にロードする
Ld	vreg(dst), data(src)	src のメモリの内容を dst にロードする
Nop		何もしない
Or	vreg(op1), data(op2)	op1 = op1 — op2
Pop	vreg(dst)	スタックから値をポップして dst に格納する
Prologue		フレームの準備を行う
Push	address(src)	src の値をスタックにプッシュする
Ret		サブルーチンからリターンする
Sl	vreg(op1), data(op2)	論理左シフトを行う
Sr	vreg(op1), data(op2)	論理右シフトを行う
St	data(dst), vreg(src)	src の値を dst にストアする
Sub	vreg(op1), data(op2)	op1 = op1 - op2
Xor	vreg(op1), data(op2)	op1 = op1 ^ op2

オペランド

TIR で定義するオペランドを表 2 に示す.

レジスタの割当て

TIR から CASL を生成する際, 仮想レジスタから物理レジスタへの割り当てを行う. 今回の設計では, 自由に使えるレジスタは GR0,GR3,GR4,GR6,GR7 の 5 つであり, かつ GR0 はアドレスのオフセットに使用することができないという制限がある. 自由に使えるレジスタは, プログラムの機能を実現するためには十分であるが, 割り当ての工夫を行うことで, ロードやストアの回数を減らすなどの最適化を行うことができる. そのため, 今回の設計では, グラフ彩色アルゴリズムを用いてレジスタ割り当てを行う.

グラフ彩色アルゴリズムとは, グラフで表現される要素に何らかの制約条件を満たすように色を塗るアルゴリズムである. 今回は, 仮想レジスタの生存期間を干渉グラフとして表現し, 互いに干渉する仮想レジスタには異なる色 (実レジスタ) を割り当てることを目指す. 例えば,

```
vreg1 = 1
vreg2 = 2
vreg3 = 3
vreg4 = vreg1 + vreg2
```

```
vreg5 = vreg3 * vreg4
vreg6 = vreg2 - vreg5
```

というコードがあった場合, 各仮想レジスタの生存期間は次のようになる.

vreg1: 行 1-行 4; vreg2: 行 2-行 6; vreg3: 行 3-行 5; vreg4: 行 4-行 5; vreg5: 行 5-6; vreg6: 行 6;

この場合, 干渉グラフは図 3 のようになる.

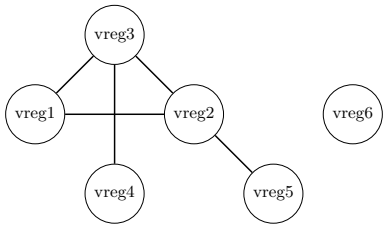


図 3: 干渉グラフの例

このグラフに対してグラフ彩色アルゴリズムを適用し, 各ノードに色を割り当てることで, 仮想レジスタから物理レジスタへの割り当てを行う. なお, 実レジスタの数に対して, 仮想レジスタの数が多い場合, スピルが発生する可能性がある. スピルとは, 割り当てられなかった仮想レジスタをメモリに退避させることであり, その場合, 仮想レジスタを使用したり更新したりする際に, ロードやストア命令を追加する必要がある. このとき, ロードやストアのたびに仮想レジスタを作り直すことで, 仮想レジスタの生存期間が短くなる. スピル後に再度グラフ彩色アルゴリズムを適用し, 再度レジスタ割り当てを試み, これを繰り返すことで, 最終的に全ての仮想レジスタに実レジスタを割り当てることを目指す. 先ほどの例で, スピルが発生し,vreg3 がメモリに退避された場合, 次のようなコードになる.

```
vreg1 = 1
vreg2 = 2
vreg3_s1 = 3
store vreg3_s1 -> mem_spill_vreg3
vreg4 = vreg1 + vreg2
load mem_spill_vreg3 -> vreg3_s2
vreg5 = vreg3_s2 * vreg4
```

なお, どの仮想レジスタをスピルするかの選択には様々な戦略が存在するが, 今回は事前に設定した優先度に基づいて決定することとした. 優先度は基本的にはすべての仮想レジスタが同じになっているが, スピルによって新たに生成された仮想レジスタについては, 優先度を下げることで再度スピルされにくくすることとした.

制御フローグラフ解析

制御フローグラフ (CFG: Control Flow Graph) 解析を行う. 制御フローグラフとは, プログラムの基本ブロック間の制御の流れを表現したグラフであり, 各ノードが基本ブロックを表し, エッジが制御の流れを表す. 制御フローグラフ解析を行い, 各基本ブロックの先行ブロックや後続ブロックを特定することで, データフロー解析や最適化に利用する. 制御フローグラフを得ることで, 仮想レジスタの生存区間解析が行えるようになり, これに基づいて干渉グラフを構築することが可能になる.

制御フローグラフの基本単位となる基本ブロックは, 次のように定義される.

1. プログラムの開始点またはジャンプ命令の直後に始まる.
2. ジャンプ命令またはプログラムの終了点で終わる.
3. ブロック内にはジャンプ命令が存在しない.

4. ブロック内の命令は, 常に順次実行される (ラベルが存在しない).

また, 基本ブロック間の制御の流れは, ジャンプ命令や条件分岐命令によって特定される. 例えば, ある基本ブロックが無条件ジャンプ命令を持つ場合, そのジャンプ先の基本ブロックが後続ブロックとなる. 条件分岐命令を持つ場合, 条件が真の場合と偽の場合の両方のジャンプ先の基本ブロックが後続ブロックとなる. 制御フローグラフは様々な用途で利用することができる. そのため, 制御フローグラフの解析は,TIR だけではなく IR やアセンブリに対しても行うことができるようにする.

3.4 IR から TIR への変換

TIR は機械語依存のため,IR では存在する命令でも,TIR では存在しない命令がある. 例えば,IR では乗算を表す命令が存在するが,TIR では乗算命令が存在しない. そのため,IR から TIR への変換時に, これらの命令を複数の TIR 命令に展開する必要がある. これらの変換について主なものを記述する.

オペランドの変換

IR のオペランドでは,Temp,Const,Variable,Label が存在するが,TIR ではこれらに対応するオペランドが異なる. IR の Temp は,TIR の VirtualRegister に対応する.IR の Const は,TIR の NumericImmediate と StringImmediate 対応する. IR の Label は,TIR の AddressOperand に対応する.IR の Variable は,TIR の FrameSlot に対応する. IR の Variable は,TIR の FrameSlot に対応する.

SSA の破壊

IR では SSA 形式を満たすように設計されているが,TIR では CASL 命令に近い形で設計されているため,SSA 形式を満たすことができない. 例えば,Add 命令は `Add op1 op2` のような形式をとり,op1 = op1 + op2 という意味を持つ. これは op1 に複数回代入が行われるため,SSA 形式を満たさない. IR では `result = op1 + op2` のように, 結果を新たな Temp に代入する形式をとるため, これを TIR に変換する際に,SSA 形式を破壊する必要がある. これは次のように変換することで実現する.

```
BinaryOp(+) result, op1, op2  -->  Add op1, op2
                                   Move result, op1
```

このような形式をとると,result にコピーせずに op1 をそのまま使うよりも仮想レジスタの生存期間が短くなるため, スピルの可能性が減少するメリットがある.

乗除算, 剰余演算

IR では `BinaryOp` 命令によって乗除算や剰余演算を表現することができるが,TIR ではこれらの命令が存在しない. これらの演算はライブラリによって行うため,TIR では CALL 命令に展開する.

比較演算

IR では `BinaryOp` 命令によって比較演算を表現することができるが,TIR ではこれらの命令が存在しない. これらの演算は,TIR の条件分岐などの複数の命令に展開する. 例えば, 等価演算は次のような TIR 命令列に展開される.

```
            Cmp op1, op2
            Jmp Zero label_true
            Lad result <- 0
            Jmp label_end
label_true: Lad result <- 1
label_end:  Nop
```

Read,Write 命令

IR では `Read`,`Write` 命令によって入出力を表現することができるが,TIR ではこれらの命令が存在しない. これらの命令もライブラリによって行うため,TIR では CALL 命令に展開する. ただし, ライブラリでは読み書きしたい値の種類によって異なるサブルーチンを呼び出す必要があるため, 読み書きする値の型に応じてサブルーチンを選択する.

Move 命令

IR では Move 命令によって変数や Temp, 定数を Temp にコピーすることができる. TIR では, コピー元が定数の場合 Lad 命令, 変数や Temp の場合は Load 命令に展開する.

3.5 最適化

3.5.1 IR の最適化

IR に対して行う大域的最適化について説明する. なお, 冗長なジャンプ命令の削除など,IR の段階で行ったほうが都合のいい局所最適化も含まれる.

定数畳み込み 定数畳み込みとは, 式の中に定数が含まれている場合, その部分を事前に計算して定数に置き換える最適化手法である. 二項演算命令について次のように置き換えを行う.

```
BinaryOp(+) result, 3, 5  -->  Move result, 8
```

定数伝播 定数伝播とは, 変数に定数が代入されている場合, その変数を使用している式に対して, 定数を直接代入する最適化手法である. 定数伝播と定数畳み込みは組み合わせて適用することで, より多くの二項演算が定数に置き換えられる可能性がある. 例えば, 次のように置き換えを行う.

```
Move tmp, 10
BinaryOp(+) result, tmp, 5  -->  BinaryOp(+) result, 10, 5
```

計算強度の低減 計算強度の低減とは, 高価な演算をより低価な演算に置き換える最適化手法である. 一般的に, 乗算や除算は加算や減算よりも高価な演算であるため, これらを置き換えることを目指す. 今回は,×2 や ÷2 の置き換えのみを対象とする. 例えば, 次のように置き換えを行う.

```
BinaryOp(*) result, var, 2  -->  BinaryOp(+) result, var, var
```

デッドコード削除 デッドコード削除とは, プログラムの実行に影響を与えない命令を削除する最適化手法である. 例えば,IF 分岐などで, 到達不可能なコードが存在する場合, そのコードを削除する. 定数畳み込みや定数伝播の結果, 条件分岐が常に真または偽になる場合, 不要な分岐先のコードを削除することができるため, これらの最適化と組み合わせて適用することで, より多くのデッドコードを削除できる可能性がある.

冗長なジャンプ命令の削除 この最適化では, すぐ次の行へのジャンプ命令を削除する. 本来このようなジャンプ命令は生成されないのだが, デッドコード削除などの最適化の過程で生成される可能性があるため, これを削除することでコードの簡素化を図る.

不要なラベルの削除 不要なラベルの削除とは, 参照されていないラベルを削除する最適化手法である. このようなラベルも, デッドコード削除などの最適化の過程で生成される可能性があるため, これを削除することでコードの簡素化を図る. 削除しなくても動作自体に影響はないが, ラベルが存在することで制御フロー解析などで必要以上にブロック分割される可能性があるため, これを防ぐ目的もある.

3.5.2 TIR の最適化

冗長なロードの削除 この最適化では, 同じフレームスロットから何度もロードされている場合, 最初のロード命令のみを残し, 以降のロード命令を削除する. 削除したロード命令使っていた仮想レジスタは, 最初のロード命令で使用していた仮想レジスタに置き換える. なお, 途中でフレームスロットが更新されている場合, その後のロード命令は削除できない. そのため, 削除するのは制御フロー解析で得られた基本ブロック内に限定する.

3.5.3 Assembly の最適化

冗長なレジスタコピーの削除 この最適化では, 同一レジスタ間のコピー命令を削除する. レジスタ割り当て時に, ロード命令でコピーされている仮想レジスタにはできるだけ同じ実レジスタを割り当てるようにすることで, このような冗長なコピー命令が生成される可能性がある. 例えば,LD GR3, GR3 のような命令は削除される.

4　実装

尺の都合上, 実装については特に工夫した部分にのみ絞り, 簡潔に説明する. 設計で説明したアルゴリズムなど他の部分の実装や詳細は git リポジトリを参照のこと.

4.1　各中間表現の実装

IR や TIR は, それぞれの命令やオペランドをクラスとして実装している. 各命令は, それぞれ IRInstr と TIRInstr という基底クラスを持ち, 共通のインターフェースを提供する. 各命令クラスは, 命令の種類ごとにサブクラスとして実装されており, 例えば,IR の二項演算命令は `class BinaryOp extends IRInstr<BinarOp>` のように実装されている. ここで,IRInstr がジェネリクスで自身の型パラメータをとっているのは,IRInstr クラスで `T withLabel(label)` のように自信を返すメソッドを実装するためである. 各オペランドも同様に,IROperand と TIROperand という基底クラスを持ち, 共通のインターフェースを提供する.

4.2　命令列の表現

最適化処理などは, 命令列を操作する必要があり, 最適化処理を行うパイプラインを共通化するためにもそれぞれの中間表現は同じ基底クラスを持つほうが良い. そのため,`IRInstr, TIRInstr, AssemblyInstr` はすべて `Instruction` という共通の基底クラスを持つように設計した. そして, 命令列は `InstructionSet<T extends Instruction>`というクラスで管理する. このクラスは, 内部的には `List<T>`で命令列を管理しており, 命令列を扱うのに特化した機能を持たせている. `InstructionSet` はそれ単体では内部の命令列を編集できないようにしている. そのため, 最適化処理などで内部の命令列を編集したい場合は,`getEditor` メソッドで `InstructionSetEditor<T>`を取得し, このクラスを通じて命令列の編集を行う. このようにすることで, 命令列の編集に関わる処理を一元化でき, 命令列の整合性を保ちながら編集を行うことができる.

`InstructionSetEditor` は, 内部に命令列のイテレータを持っており, 一部の処理をイテレータに委譲する形で命令列の編集を行う. また, 命令の置き換えや前後への追加など, 最適化処理の際に便利なメソッドを提供している. 例えば, 定数伝播の最適化において, メモリからのロードを定数の読み込みに置き換える場合, 次のように実装できる.

```
    while(editor.hasNext()) {
        var instr = editor.next();
        //中略
        if(instr instanceof Load load) {
            if(constantValues.containsKey(load.var.get())) {
                Object constValue = constantValues.get(load.var.get());
                editor.prev(); // イテレータをロード命令の位置に戻す
                editor.replace(new Move(load.targetVar.get(), new Const(constValue)));
                optimized = true;
            }
        }
    }
```

4.3　中間表現の変換処理の実装

各中間表現は,AST と同様に Visitor パターンを実装しており, 変換処理も Visitor パターンを用いて実装している. Visitor パターンを用いることで, 各命令の変換処理を命令クラス内に分散させることができ, かつ Visitor クラスに状態を持たせることで, 変換処理全体で共有する情報を管理しやすくなる.

4.4　最適化パイプラインの実装

最適化パイプラインは,`OptimizerPipeline` クラスに実装されている. このクラスには, 最適化パスを追加することができ, 最適化パスは `OptimizePass` インターフェースを実装すればよい. 各最適化パスは, 命令列を受け取り, 最適化を行い, 最適化が行われたかどうかを返す `apply` メソッドを実装する. 最適化パイプラインは, 命令列に対して複

数の最適化パスを登録された順に実行する. そして, 最適化パスを一周してすべての apply メソッドが false を返すまで繰り返し実行することで, 最適化が収束するまで最適化を行う.

5 工夫点

工夫点について説明する. なお, タスクランナーや追加テストなどはこれまでの課題と同様に継続しているため過去のレポートを参照のこと.

5.1 CASL 以外の機械語への拡張性の確保

今回実装したコンパイラは,CASL 以外の機械語への拡張性を持たせている. 例えば, `TIRInstr` は `AnalyzableInstr` という基底クラスを持っており, 制御フローグラフや生存区間解析はこの基底クラスを対象に動作するようにしている. また, `VirtualRegister` は `AllocatableRegister` という基底クラスを持っており, 干渉グラフはこのクラスを対象に動作する. このように, 対象コードの生成処理に関わる様々な部分で抽象化を図っているため, これらの基底クラスやインターフェースを実装した中間表現やオペランドを用意することでこれらのインフラを利用できる. このことにより,CASL 以外の機械語に対応することが容易になる.

このことを実証するため, 情報科学実験 C の授業で作成する C-Processor 向けのアセンブリ (以後アセンブリ) を生成するように TIR 以降の処理を c-processor ブランチに実装してみた. この実装においては, 制御フローグラフ, 干渉グラフ, 生存区間解析, レジスタ割り当てなど多くのインフラを利用することができたため, 基本的には TIR や Assembly クラスの実装と, それらの間の変換処理などを追加するだけで簡単に実現できた. なお, 局所最適化などの実装は行っていない. このアセンブリは,CASL に比べてかなり簡素であり, 汎用レジスタは 2 個しかないうえスタックも利用することができない. また, メモリアドレスを動的に計算することもできない. これらの制約により, 副プログラム宣言は利用できないなどかなりの制約を設けることとはなったが, 最大公約数の計算プログラムなどの簡単なプログラムのコンパイルに成功し, 実際に動作することも確認できた. 参考までに, コンパイル後のアセンブリを付録に付した.

5.2 ジェネリクスの活用

今回の実装では, 命令列やそれに関わる諸クラス (レジスタ割り当て関連, 最適化関連など) でジェネリクスを活用した. ジェネリクスは, クラスやメソッドで扱うデータ型をパラメータ化することで, 型安全性を保ちながら柔軟な設計を可能にする機能である. 例えば,Assembly における 2 オペランド命令は基底として `Instr2Op<T1,T2>` を持ち, それぞれのオペランドの型をジェネリクスで指定することで, 様々な型のオペランドを持つ命令を表現できるようにした. これを利用して,ADD 命令は `class ADD extends Instr2Op<RegisterOperand, DataOperand>` のように定義することで, オペランドの型を明確にしつつ, 共通の基底クラスから継承した機能を利用できるようにした. このようにジェネリクスを活用することで, 命令列やそれに関わる諸クラスの設計がより柔軟かつ型安全になり, コードの再利用性も向上したと感じている. 一方で, あまりにもジェネリクスを使いすぎたため, 逆にコードが複雑しまったと感じる部分もあった. 適切なバランスを見極めることが重要であると感じた.

6 反省点と感想

課題として設定されたものよりもさらに高機能かつ拡張性の高いコンパイラを実装でき, とても満足している. たた, 生成されたアセンブリを見てみると, ジャンプ命令 (JMI や JZE など) の変更やレジスタ割り当ての変更などでさらに命令量を減らせる箇所がみられるため, さらなる最適化を施す余地も多くみられると感じた. 今回, 最適化を行ったが, 最適化の効果を定量的に評価できていない. 最適化前後でステップ数を計測するなど効果を検証したいと思った.

コンパイラの実装コードについては, まだまだ詰めの甘い部分は多くみられるため, コード全体のリファクタリングは必要であると感じた. テスト関連がかなり貧弱であるため, 特定のコードをコンパイルした際に意図しない動作をする可能性がある. テストが少ない状況ではリファクタリング後の品質も保証できないため, 単体テストの追加などテストの充実は急務であると感じた.

最後に, 無駄な部分が多くさらに洗練させる余地があるとはいえ,10 ページの制限では設計や実装, 工夫点などを表現しきれなかったため, レポートのページ制限の緩和をお願いしたいです.

付録.A　C-Processor 向けコンパイルの結果

付録.A.1　元のプログラム

```
program GCD;

var
a, b, result: integer;

begin
a := 48;
b := 18;

while a <> b do
begin
    if a > b then
    begin
    a := a - b;
    end
    else
    begin
    b := b - a;
    end;
end;

result := a;
end.
```

付録.A.2　出力されたアセンブリ

```
 LDI A 48
    SETIX V1
    STD A
    LDI A 18
    SETIX V2
    STD A
L1: NOP
    SETIX V1
    LDD B
    SETIX V2
    LDD A
    CMP
    JPZ BL1
    LDI A 1
    JMP BL2
BL1: NOP
    LDI A 0
BL2: NOP
    LDI B 0
    CMP
    JPZ L2
```

```
    SETIX V1
    LDD B
    SETIX V2
    LDD A
    SUB A
    LDI B 128
    AND A
    JPZ BL3
    LDI B 1
    JMP BL4
BL3: LDI B 0
BL4: NOP
    LDI A 1
    CMP
    JPZ L3
    SETIX V2
    LDD A
    SETIX V1
    LDD B
    SUB A
    MOV B
    SETIX V2
    STD B
    JMP L4
L3: NOP
    SETIX V1
    LDD A
    SETIX V2
    LDD B
    SUB A
    MOV B
    SETIX V1
    STD B
L4: NOP
    JMP L1
L2: NOP
    SETIX V1
    LDD A
    SETIX V3
    STD A
V2: DS 1
V1: DS 1
V3: DS 1
```